

Deep Learning

Lec 9

Transfer learning

Intro...

- Transfer learning is one of the most important techniques of deep learning.
- When building a vision system to solve a specific problem, you usually need to collect and label a huge amount of data to train your network.
- You can build convnets, as you learned and start the training from scratch; that is an acceptable approach.
- But what if you could download an existing neural network that someone else has tuned and trained, and use it as a starting point for your new task?

Transfer learning allows you to do just that.

- You can download an open source model that someone else has already trained and tuned and use their optimized parameters (weights) as a starting point to train your model on a smaller dataset for a given task. This way, you can train your network a lot faster and achieve higher results.

Let ask ourselves some questions

- What problems does transfer learning solve?
- As the name implies, transfer learning means transferring what a neural network has learned from being trained on a specific dataset to another related problem.
- Transfer learning is currently very popular in the field of DL because it enables you to train deep neural networks with comparatively **little data** in a short training time.
- The importance of transfer learning comes from the fact that in most real-world problems, we typically do not have millions of labeled images to train such complex models.

The idea is pretty straightforward.

- First we train a deep neural network on a very large amount of data.
- During the training process, the network extracts a large number of useful features that can be used to detect objects in this dataset. We then transfer these extracted features (feature maps) to a new network and train this new network on our new dataset to solve a different problem.
- Transfer learning is a great way to shortcut the process of collecting and training huge amounts of data simply by reusing the model weights from pre-trained models that were developed for standard CV benchmark datasets, such as the ImageNet image-recognition tasks. Top-performing models can be downloaded and used directly, or integrated into a new model for your own CV problems.
- Additionally, an important benefit of using transfer learning is that it helps the model generalize its learnings and avoid overfitting.

- why would we want to use transfer learning? Why don't we just train a neural network directly on our new dataset to solve our problem?

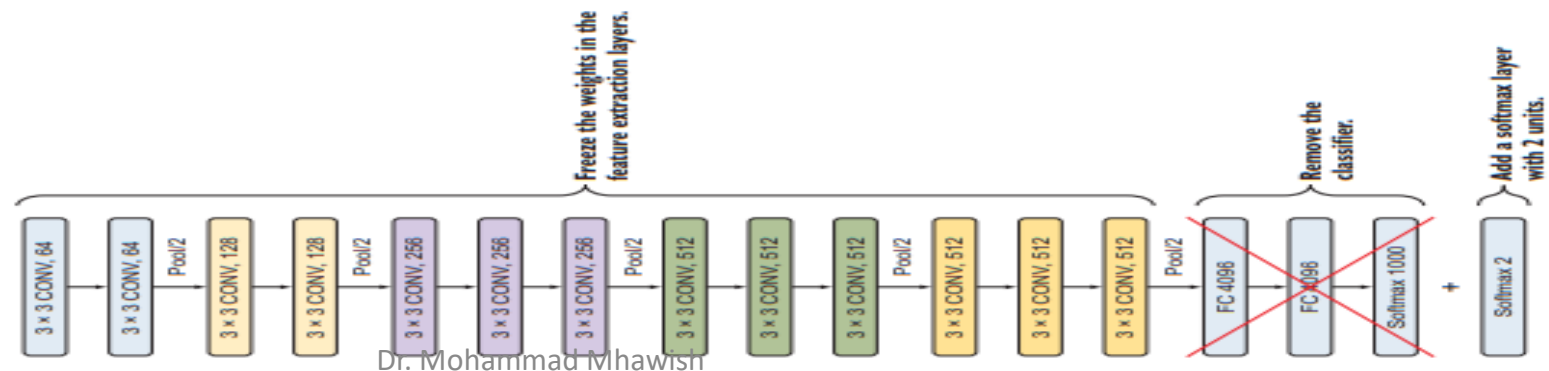
Deep neural networks are immensely **data-hungry** and rely on huge amounts of labeled data to achieve high performance. In practice, very few people train an entire convolutional network from scratch. This is due to two main problems:

- Data problem—Training a network from scratch requires a lot of data in order to get decent results, which is not feasible in most cases. It is relatively rare to have a dataset of sufficient size to solve your problem. It is also very expensive to acquire and label data: this is mostly a manual process done by humans capturing images and labeling them one by one, which makes it a nontrivial task.
- Computation problem—Even if you are able to acquire hundreds of thousands of images for your problem, it is computationally very expensive to train a deep neural network on millions of images because doing so usually requires weeks of training on multiple GPUs. Also keep in mind that training a neural network is an iterative process. So, even if you happen to have the computing power required to train a complex neural network, spending weeks experimenting with different hyper-parameters in each training iteration until you finally reach satisfactory results will make the project very costly.

What is transfer learning?

- Transfer learning is the transfer of the knowledge (feature maps) that the network has acquired from one task, where we have a large amount of data, to a new task where data is not abundantly available. It is generally used where a neural network model is first trained on a problem similar to the problem that is being solved. One or more layers from the trained model are then used in a new model trained on the problem of interest.
- Knowing that this would be a problem for people with little-to-no resources, researchers built state-of-the-art models that were trained on large image datasets like ImageNet, MS COCO, Open Images, and so on, and then shared their models with the general public for reuse. This means you should never have to train an image classifier from scratch again, unless you have an exceptionally large dataset and a very large computation budget to train everything from scratch by yourself. Even if that is the case, you might be better off using transfer learning to fine-tune the pre-trained network on your large dataset.
- The intuition behind transfer learning is that if a model is trained on a large and general enough dataset, this model will effectively serve as a generic representation of the visual world. We can then use the feature maps it has learned, without having to train on a large dataset, by transferring what it learned to our model and using that as a base starting model for our own task.

- What you have to do to use transfer learning?
- Suppose we want to train a model that classifies dog and cat images, and we have only two classes in our problem: dog and cat.
- First, we need to find a dataset that has **similar features to our problem** at hand. This involves spending some time exploring different open source datasets to find the one closest to our problem. For the sake of this example, let's use **ImageNet**, it has a lot of dog and cat images. So the pre-trained network is familiar with dog and cat features and will require minimum training.
- Next, we need to choose a network that has been trained on **ImageNet** and achieved good results. In last lecture, we learned about state-of-the-art architectures like VGGNet, GoogLeNet, and ResNet. Any of them would work fine. For this example, we will go with a VGG16 network that has been trained on ImageNet datasets.
- To adapt the VGG16 network to our problem, we are going to download it with the pretrained weights, remove the classifier part, add our own classifier, and then retrain the new network. This is called using a **pretrained network as a feature extractor**.



How use transfer learning :

1. Download the open source code of the VGG16 network and its weights to create our base model, and remove the classification layers from the VGG network (FC_4096 > FC_4096 > Softmax_1000):
 - <https://keras.io/api/applications>

```
from keras.applications.vgg16 import VGG16
```

Imports the VGG16
model from Keras

```
base_model = VGG16(weights = "imagenet", include_top=False,  
                    input_shape = (224,224, 3))  
base_model.summary()
```

Notice that this downloaded architecture does not contain the classifier part (three fully connected layers) at the top of the network because we set the `include_top` argument to `False`

Downloads the model's pretrained weights and saves them in the variable `base_model`. We specify that Keras should download the ImageNet weights. `include_top` is false to ignore the fully connected classifier part on top of the model.

```
Total params: 14,714,688
```

```
Trainable params: 14,714,688
```

```
Non-trainable params: 0
```


2. Freeze the feature extraction layers that have been trained on the ImageNet dataset. Freezing layers means freezing their trained weights to prevent them from being retrained when we run our training:

```
for layer in base_model.layers:  
    layer.trainable = False  
  
base_model.summary()
```

← Iterates through layers
and locks them to make them
non-trainable with this code

```
Total params: 14,714,688  
Trainable params: 0  
Non-trainable params: 14,714,688
```

The model summary is omitted in this case for brevity, as it is similar to the previous one. The difference is that all the weights have been frozen, the trainable parameters are now equal to zero, and all the parameters of the frozen layers are non-trainable:

- 3. Add our own classification dense layer. Here, we will add a softmax layer with two units because we have only two classes in our problem

```

from keras.layers import Dense, Flatten
from keras.models import Model

last_layer = base_model.get_layer('block5_pool')
last_output = last_layer.output

```

Imports Keras modules

Uses the get_layer method to save the last layer of the network

Saves the output of the last layer to be the input of the next layer

```

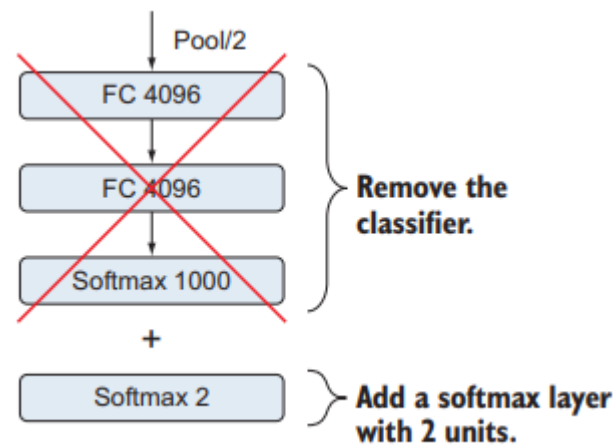
x = Flatten()(last_output)

x = Dense(2, activation='softmax', name='softmax')(x)

```

Flattens the classifier input, which is the output of the last layer of the VGG16 model

Adds our new softmax layer with two hidden units



4. Build a new_model that takes the input of the base model as its input and the output of the last softmax layer as an output. The new model is composed of all the feature extraction layers in VGGNet with the pretrained weights, plus our new, untrained, softmax layer. In other words, when we train the model, we are only going to train the softmax layer in this example to detect the specific features of our new problem (German Shepherd, Beagle, Neither):

```
new_model = Model(inputs=base_model.input, outputs=x)
new_model.summary()
```

Instantiates a new_model using Keras's Model class

Prints the new_model summary

```
=====
Total params: 14,789,955
Trainable params: 50,178
Non-trainable params: 14,714,688
=====
```

- Training the new model is a lot faster than training the network from scratch. To verify that, look at the number of trainable params in this model (~50,000) compared to the number of non-trainable params in the network (~14 million). These “nontrainable” parameters are already trained on a large dataset, and we froze them to use the extracted features in our problem.
- With this new model, we don’t have to train the entire VGGNet from scratch because we only have to deal with the newly added softmax layer.
- Additionally, we get much better performance with transfer learning because the new model has been trained on millions of images (ImageNet dataset + our small dataset). This allows the network to understand the finer details of object nuances, which in turn makes it generalize better on new, previously unseen images.

some recap questions

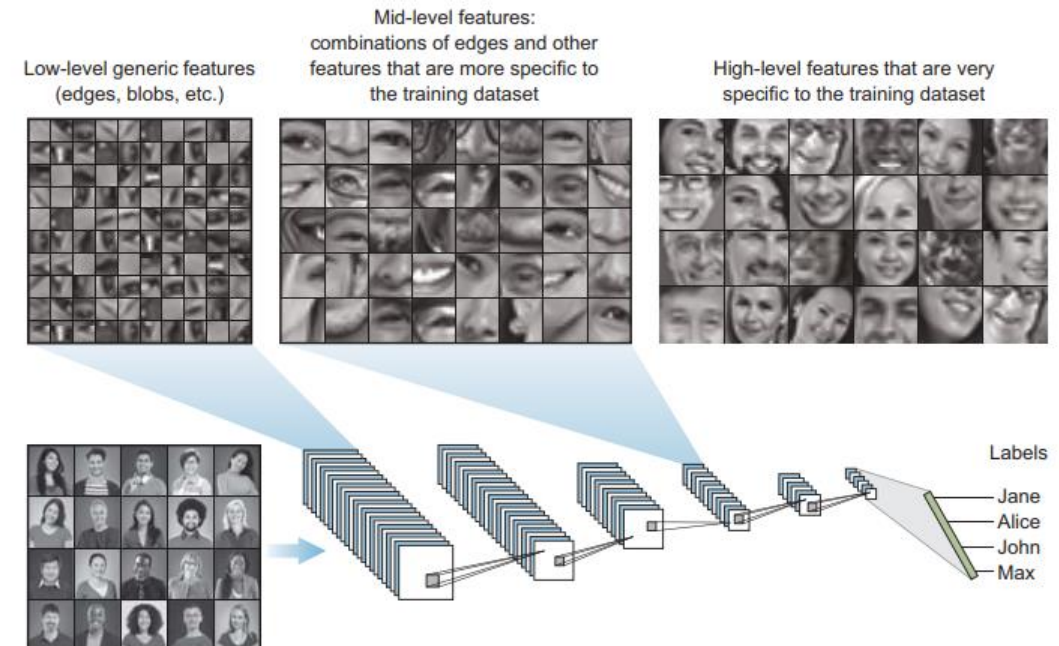
- What is really being learned by the network during training?
 - The short answer is: feature maps.
- How are these features learned?
 - During the backpropagation process, the weights are updated until we get to the optimized weights that minimize the error function.
- What is the relationship between features and weights?
 - A feature map is the result of passing the weights filter on the input image during the convolution process
- What is really being transferred from one network to another?
 - To transfer features, we download the optimized weights of the pretrained network. These weights are then reused as the starting point for the training process and retrained to adapt to the new problem.

- Okay, let's dive into the details to understand what we mean when we say **pre-trained network**.
- When we're training a convolutional neural network, the network extracts features from an image in the form of feature maps: outputs of each layer in a neural network after applying the weights filter.
- Now, recall that neural networks iteratively update their weights during the training cycle of feedforward and backpropagation. We say the network has been trained when we go through a series of training iterations and hyperparameter tuning until the network yields satisfactory results.
- When training is complete, **we output two main items: the network architecture and the trained weights**. So, when we say that we are going to use a pre-trained network, we mean that we will download the **network architecture together with the weights**.
- During training, the model learns only the features that exist in this training dataset. But when we download large models (like Inception) that have been trained on huge numbers of datasets (like ImageNet), all the features that have already been extracted from these large datasets are now available for us to use.
- I find that really exciting because these pre-trained models have spotted other features that weren't in our dataset and will help us build better convolutional networks

How do neural networks learn features?

- The deeper you go through the network layers, the more image-specific features are learned. In figure 6.6, the first layer detects low-level features such as edges and curves.
- The output of the first layer becomes input to the second layer, which produces higher-level features like semicircles and squares. The next layer assembles the output of the previous layer into parts of familiar objects, and a subsequent layer detects the objects.
- As we go through more layers, the network yields an activation map that represents more complex features.
- As we go deeper into the network, the filters begin to be more responsive to a larger region of the pixel space. Higher-level layers amplify aspects of the received inputs that are important for discrimination and suppress irrelevant variations.

The transferability of features that are extracted at later layers depends on the similarity of the original and new datasets. The idea is that all images must have shapes and edges, so the early layers are usually transferable between different domains. We can only identify differences between objects when we start extracting higher-level features: say, the nose on a face or the tires on a car



The source domain is the original dataset that the pretrained network is trained on. The target domain is the new dataset that we want to train the network on.

Transfer learning approaches

- **Using a pretrained network as a classifier... how?**

- Using a pretrained network as a classifier doesn't involve freezing any layers or doing extra model training. Instead, we just take a network that was trained on a similar problem and deploy it directly to our task. The pretrained model is used directly to classify new images with no changes applied to it and no extra training. All we do is download the network architecture and its pretrained weights and then run the predictions directly on our new data. In this case, we are saying that the domain of our new problem is very similar to the one that the pretrained network was trained on, and it is ready to be deployed.

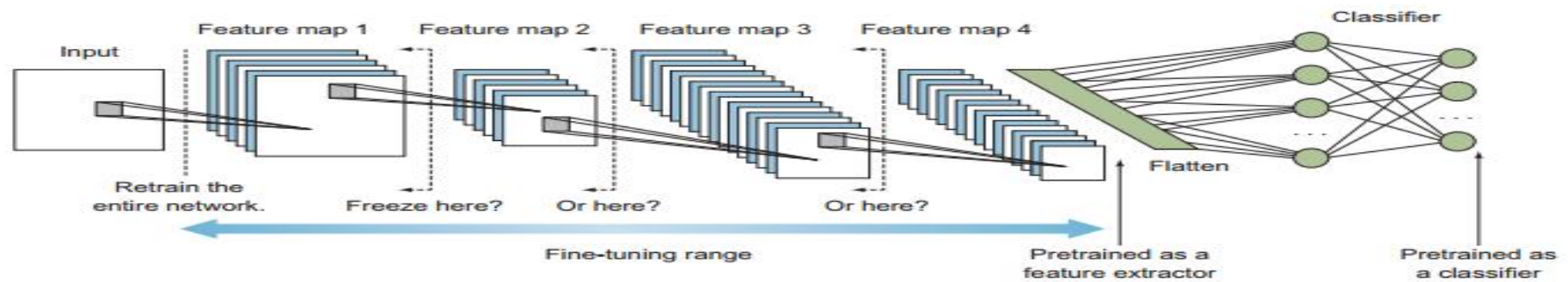
```
model = VGG16(weights = "imagenet", include_top=True, input_shape =  
              (224,224, 3))
```

- **Using a pretrained network as a feature extractor... how?**

- we take a pretrained CNN on ImageNet, freeze its feature extraction part, remove the classifier part, and add our own new, dense classifier layers.

fine-tuning

- The **formal definition of fine-tuning** is freezing a few of the network layers that are used for feature extraction, and jointly training both the non-frozen layers and the newly added classifier layers of the pretrained model.
- It is called fine-tuning because when we retrain the feature extraction layers, we fine-tune the higher-order feature representations to make them more relevant for the new task dataset.
- feature maps that are extracted early in the network are **generic**. The feature maps get progressively **more specific** as we go deeper in the network. This means feature maps 4 in figure below are very specific to the source domain. Based on the similarity of the two domains, we can decide to freeze the network at the appropriate level of feature maps:
- If the domains are similar, we might want to freeze the network up to the last feature map level (feature maps 4, in the example).
- If the domains are very different, we might decide to freeze the pretrained network after feature maps 1 and retrain all the remaining layers.
- Between these two possibilities are a range of fine-tuning options that we can apply. We can retrain the entire network, or freeze the pretrained network at any level of feature maps 1, 2, 3, or 4 and retrain the remainder of the network. We typically decide the appropriate level of fine-tuning by trial and error.



- WHY IS FINE-TUNING BETTER THAN TRAINING FROM SCRATCH?

When we train a network from scratch, we usually randomly initialize the weights and apply a gradient descent optimizer to find the best set of weights that optimizes our error function.

Since these weights start with random values, there is no guarantee that they will begin with values that are close to the desired optimal values. And if the initialized value is far from the optimal value, the optimizer will take a long time to converge.

This is when fine-tuning can be very useful. The pretrained network's weights have been already optimized to learn from its dataset. Thus, when we use this network in our problem, we start with the weight values that it ended with. So, the network converges much faster than if it had to randomly initialize the weights.

Choosing the appropriate level of transfer learning

choosing the appropriate level for transfer learning is a function of two important factors:

- ***Size of the target dataset (small or large)*** —When we have a small dataset, the network probably won't learn much from training more layers, so it will tend to overfit the new data. In this case, we most likely want to do less fine-tuning and rely more on the source dataset.
- ***Domain similarity of the source and target datasets***—How similar is our new problem to the domain of the original dataset? For example, if your problem is to classify cars and boats, ImageNet could be a good option because it contains a lot of images of similar features. On the other hand, if your problem is to classify lung cancer on X-ray images, this is a completely different domain that will likely require a lot of fine-tuning.
- **These two factors lead to the four major scenarios:**
 - **1** The target dataset is *small* and *similar* to the source dataset.
 - **2** The target dataset is *large* and *similar* to the source dataset.
 - **3** The target dataset is *small* and *very different* from the source dataset.
 - **4** The target dataset is *large* and *very different* from the source dataset.

- **Scenario 1: Target dataset is small and similar to the source dataset**
 - Since the original dataset is similar to our new dataset, we can expect that the higherlevel features in the pretrained ConvNet are relevant to our dataset as well. Then it might be best to freeze the feature extraction part of the network and only retrain the classifier.
- **Scenario 2: Target dataset is large and similar to the source dataset**
 - Since both domains are similar, we can freeze the feature extraction part and retrain the classifier, similar to what we did in scenario 1. But since we have more data in the new domain, we can get a performance boost from fine-tuning through all or part of the pretrained network with more confidence that we won't overfit. So a good start is to freeze approximately 60–80% of the pretrained network and retrain the rest on the new data.
- ***Scenario 3: Target dataset is small and different from the source dataset***

Since the dataset is different, it might not be best to freeze the higher-level features of the pretrained network, because they contain more dataset-specific features.

A good start is to freeze approximately the first third or half of the pretrained network. After all, the early layers contain very generic feature maps that will be useful for your dataset even if it is very different.
- **Scenario 4: Target dataset is large and different from the source dataset**

In this case, we have a large dataset that provides us with the confidence to fine-tune through the entire network without having to worry about overfitting . Or start from scratch

Scenario	Size of the target data	Similarity of the original and new datasets	Approach
1	Small	Similar	Pretrained network as a feature extractor
2	Large	Similar	Fine-tune through the full network
3	Small	Very different	Fine-tune from activations earlier in the network
4	Large	Very different	Fine-tune through the entire network

